HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN

BỘ MÔN LẬP TRÌNH PYTHON

BÁO CÁO BÀI TẬP LỚN

Giảng viên hướng dẫn	**: KIM NGỌC BÁCH**
Họ và tên sinh viên	**: VŨ MINH SÁNG**
Mã sinh viên	**: B23DCCE082**
Lớp	**: D23CQCE04-B**
Nhóm	**: 04**

Hà Nội – 2025

Table of Contents

Overview and Objectives

This assignment challenges students to leverage the **Python programming language** to collect, process, and analyze football player statistics from the **2024–2025 English Premier League season**. It offers a hands-on experience in data analysis, specifically in the domain of **sports analytics**, where data-driven decisions are crucial for assessing player performance and making strategic choices.

Students will begin by gathering data from a football statistics website (fbref.com) and addressing common data-related issues, such as missing or inconsistent values. They will learn how to clean and structure raw data, transforming it into a usable format for further analysis. This process will involve working with a wide array of player statistics, which include metrics such as goals, assists, and playing time.

After data cleaning and preprocessing, students will perform various statistical analyses, such as calculating **averages, medians**, and **standard deviations**, to identify standout or underperforming players across different metrics. They will also explore the distribution of statistics through **histograms**, gaining a better understanding of performance trends within the league and among different teams.

To deepen their analytical skills, students will apply **machine learning** techniques like **K-means clustering** to group players based on their performance characteristics. In addition, they will use **Principal Component Analysis (PCA**) to reduce data dimensions, enabling a clearer, two-dimensional representation of player clusters for easy interpretation.

Another key component of the assignment is collecting player **transfer market values** from another source (footballtransfers.com) and analyzing them in relation to player performance data. Students will be required to propose a method

for estimating player market values, selecting appropriate features and models to develop a predictive model for this purpose.

The primary objective of this assignment is to help students develop a comprehensive understanding of **data manipulation**, **statistical analysis**, and **machine learning** within the context of real-world sports data. The skills gained throughout the project such as **data cleaning, statistical reasoning**, and **modeling techniques** are vital for anyone pursuing a career in **data science, sports analytics,** or **software development**.

1. Data Collection from FBref

1.1. Objective

Gather comprehensive statistical data for players in the 2024–2025 Premier League season from **FBref.com**, specifically for those who have played more than 90 minutes.

1.2. Tools used

Libraries and Modules:

Library/Module	Type	Purpose
pandas	Third-party	Provides high-performance data structures like DataFrames for structured data analysis and manipulation.
undetected _chromedriver (as uc)	Third-party	A Selenium wrapper that allows Chrome automation while bypassing bot-detection mechanisms.
bs4 (BeautifulSoup)	Third-party	Used for parsing and navigating HTML/XML content to extract data from web pages.
time	Standard Library	Handles time-related functions, such as sleep delays between actions to mimic human behavior.
utils	User-defined	Contains helper functions, such as `remove_duplicate_column`, used for cleaning and handling duplicated columns in datasets.

1.3. Temporarily Disabled Tools Due to CAPTCHA Issues

These tools were part of the original implementation but were commented out due to CAPTCHA protection interfering with automated access:

Library/Module	Type	Purpose
selenium.webdriver.common.by.By	Third-party	Provides methods to locate web page elements (by ID, class name, CSS selector, etc.).
selenium.webdriver.support.ui.WebDriverWait	Third-party	Explicitly waits for certain conditions (e.g., element to appear) before proceeding.
selenium.webdriver.support.expected_conditions	Third-party	Used in combination with WebDriverWait to define expected conditions (e.g., element is clickable).
webdriver_manager.chrome.ChromeDriverManager	Third-party	Automatically handles downloading and setup of the appropriate version of ChromeDriver.

1.4. Summary of Methodology

The methodology for scraping and processing Premier League player statistics from the fbref.com website involves several key steps:

- **Table Identification**: In the initial step, a custom function in the **utils** module is used to identify the unique IDs of the tables containing the required player statistics. This function scans the HTML content and locates the specific table IDs necessary for further data extraction. These table IDs, along

with their corresponding URLs, are stored in a dictionary for easy access during the scraping process.

- **Web Scraping Setup**: The code utilizes **undetected_chromedriver** and **selenium** to automate browser interaction, effectively bypassing anti-bot mechanisms on the website. Chrome is configured with options such as disabling GPU acceleration and sandboxing to ensure smooth operation in headless mode, enabling efficient scraping without manual intervention.
- **Data Extraction**: The script targets multiple URLs, each containing detailed statistical tables for different player categories, such as standard stats, shooting, passing, goalkeeping, defense, and more. The **BeautifulSoup** library is used to parse the HTML content and locate the relevant tables by their unique IDs (stored in the dictionary). Data is then extracted from each table, ensuring consistency in formatting.
- **Data Cleaning and Transformation**: To ensure consistency and accuracy, columns such as "**Min**" (minutes played) are cleaned by removing commas and converting them into numeric values. **Age** values are processed by calculating the average of a player's age range to provide a more consistent representation. Additionally, players with less than 90 minutes played are filtered out to retain only relevant data, ensuring the dataset focuses on players with significant participation.
- **Data Consolidation**: The extracted data from multiple tables is merged into a single DataFrame, combining shared columns such as **Player**, **Nation**, **Position**, and **Squad**. This consolidation allows for a comprehensive dataset that includes all relevant performance metrics. Each table's columns are prefixed with category-specific labels (e.g., **Standard_**, **Goalkeeping_**) to maintain clarity in distinguishing between different types of statistics.
- **Column Renaming and Final Processing**: The column names are standardized and cleaned to improve readability and to ensure they align with conventional naming standards. Missing or incomplete values are handled by

replacing them with "**N/a**", maintaining the integrity and completeness of the dataset. Finally, the dataset is sorted alphabetically by player name to facilitate easy navigation and analysis.

- **Output**: The final, cleaned, and merged dataset is saved into a CSV file named "**results.csv**". This file contains all the relevant player statistics, organized across various performance categories, serving as the final output for further analysis.

1.5. Results Obtained

results.csv

Player	Nation	Pos	Team	Age	Standard_MP	Standard_Starts	Standard_Min	Standard_Gls	Standard_Ast	Standard_CrdY	Standard_CrdR	Standard_xG	St
Aaron Cresswell	ENG	DF	West Ham	35.36	14	7	589.0	0	0	2	0	0.1	1.
Aaron Ramsdale	ENG	GK	Southampton	26.95	25	25	2250.0	0	0	2	0	0.0	0.
Aaron Wan-Bissaka	ENG	DF	West Ham	27.41	31	30	2704.0	2	2	1	0	1.0	2.
Abdoulaye Doucouré	MLI	MF	Everton	32.31	29	28	2335.0	3	1	5	1	3.7	2.
Abdukodir Khusanov	UZB	DF	Manchester City	21.15	6	6	503.0	0	0	1	0	0.0	0.
Abdul Fatawu Issahaku	GHA	FW	Leicester City	21.13	11	6	579.0	0	2	0	0	0.4	1.
Adam Armstrong	ENG	FW,MF	Southampton	28.2	20	15	1248.0	2	2	4	0	3.3	1.
Adam Lallana	ENG	MF	Southampton	36.96	14	5	361.0	0	2	4	0	0.2	0.
Adam Smith	ENG	DF	Bournemouth	33.99	21	16	1319.0	0	0	5	0	0.7	0.
Adam Webster	ENG	DF	Brighton	30.3	11	8	617.0	0	0	0	0	0.0	0.
Adam Wharton	ENG	MF	Crystal Palace	20.9	19	15	1258.0	0	2	2	0	0.3	3.
Adama Traoré	ESP	FW,MF	Fulham	29.25	31	16	1523.0	2	5	2	0	3.8	4.
Albert Grønbaek	DEN	FW,MF	Southampton	23.92	4	2	143.0	0	0	0	0	0.1	0.
Alejandro Garnacho	ARG	MF,FW	Manchester Utd	20.82	32	21	1966.0	5	1	2	0	6.2	3.
Alex Iwobi	NGA	FW,MF	Fulham	28.98	33	31	2636.0	9	5	1	0	4.3	6.
Alex McCarthy	ENG	GK	Southampton	35.39	5	5	450.0	0	0	0	0	0.0	0.
Alex Palmer	ENG	GK	Ipswich Town	28.71	9	9	810.0	0	0	2	0	0.0	0.
Alex Scott	ENG	MF	Bournemouth	21.68	16	6	612.0	0	0	2	0	0.7	0.
Alexander Isak	SWE	FW	Newcastle Utd	25.59	30	30	2411.0	21	6	1	0	17.5	4.
Alexis Mac Allister	ARG	MF	Liverpool	26.33	32	29	2471.0	4	4	6	0	2.7	4.
Ali Al Hamadi	IRQ	FW	Ipswich Town	23.15	11	0	134.0	0	0	3	0	0.4	0.
Alisson	BRA	GK	Liverpool	32.56	23	23	2058.0	0	0	0	0	0.0	0.
Alphonse Areola	FRA	GK	West Ham	32.16	22	21	1900.0	0	0	0	0	0.0	0.
Amad Diallo	CIV	FW,MF	Manchester Utd	22.79	22	17	1594.0	6	6	3	0	4.1	3.
Amadou Onana	BEL	MF	Aston Villa	23.69	22	17	1348.0	3	0	4	0	2.1	0.
Andreas Pereira	BRA	MF	Fulham	29.31	30	22	1834.0	2	4	7	0	3.4	4.
Andrew Robertson	SCO	DF	Liverpool	31.12	30	26	2218.0	0	0	3	1	0.9	4.
André	BRA	MF	Wolves	23.78	28	26	2087.0	0	0	7	0	0.3	0.

Overview of results.csv

The result obtained in the result.csv file includes all the columns as specified in the assignment. All missing values have been replaced with "**N/a**" and the players are sorted based on their first names. While the columns could be further organized using a **MultiIndex** for better grouping, such as categorizing the statistics under headers like **Standard, Goalkeeping, Shooting**, and so on, this approach was not implemented due to time constraints.

2. Statistical Data Analysis

2.1. Objective

The objective of this problem is to analyze Premier League player statistics by identifying top and bottom performers, calculating statistical metrics (**median**, **mean**, and **standard deviation**) for each statistic, visualizing data through histograms, and determining which team is performing the best in the 2024-2025 season based on these analyses. Results are saved in "**top_3.txt**" and "**results2.csv**" for further review.

2.2. Tools used:

Library/Module	**Type**	**Purpose**
pandas	Third-party	Provides high-performance data structures like **DataFrames** for structured data analysis and manipulation.
from prettytable import PrettyTable	Third-party	A library for displaying tabular data in a visually appealing ASCII table format. Imports the **PrettyTable** class from the **prettytable** library.
matplotlib.pyplot	Third-party	A module within the **matplotlib** library used for creating static, animated, and interactive visualizations, specifically for plotting graphs.

2.3. Summary of Methodology

2.3.1. **Top and Bottom 3 player of each category**

- **Read the CSV**: The data from the file **"results.csv"** is loaded into a pandas DataFrame using the **pd.read_csv()** function.
- **Extract Relevant Columns:** The columns that represent player statistics are selected by excluding non-statistics columns such as **Player, Position, Team**, and **Nation.**
- **Convert to Numeric**: Each statistic column is converted to a numeric format using the **apply(pd.to_numeric, errors='coerce')** function, which automatically converts any non-numeric values to **NaN**.
- **Find Top and Bottom Players:** The **nlargest()** and **nsmallest()** functions are used to identify the top 3 and bottom 3 players for each statistical category. This helps to highlight the best and worst performers.
- **PrettyTable Output**: A **PrettyTable** is generated to format the top 3 and bottom 3 players for each statistic. The table is then saved to a file named "**top_3.txt**", making it easy to read and share.

2.3.2. **Calculate median, mean, and standard deviation for each statistic, then save to 'results2.csv'.**

- **Read the Data**: We use **pd.read_csv("result.csv")** to load the data from the CSV file into a pandas DataFrame.
- **Generate Columns:** The **make_header()** function generates the column names for the DataFrame, including statistics like **Goals, Assists, etc**. Modify this function based on the actual columns in your data.

- **Process Player Data**: We loop through each row in the **DataFrame** (**data.iterrows()**), extracting the relevant player statistics and appending them to the new DataFrame df.
- **Group by Team**: The **groupby('Team')** method is used to group the data by teams, and then the agg() method calculates the median, mean, and standard deviation for each statistic.
- **Calculate Statistics**: For each team and each statistic (Goals, Assists, Passes, etc.), the median, mean, and standard deviation are calculated using pandas methods **.median()**, **.mean()**, and **.std()**.
- **Add to DataFrame**: The calculated statistics for each team are added to a new row, and that row is appended to the DataFrame.
- **Save to CSV**: Finally, the **to_csv()** method saves the DataFrame to "**results2.csv**", which includes both individual player statistics and team statistics.

2.3.3. **Plot histograms for each statistic, showing distribution for all players and each team.**

- **Read Data**: Load the dataset from a CSV file named "**results.csv**" into a pandas DataFrame.
- **Identify Criteria**: List the criteria (columns) needed for further analysis.
- **Handle Missing Data**: Replace the string "**N/a**" in the dataset with **NaN** (Not a Number) to handle missing values appropriately.
- **Convert to Numeric**: For each column, convert all values to numeric, while ensuring that non-numeric values (such as text) are turned into **NaN**.
- **Plot Histograms**: After cleaning the data, generate histograms for each column, visualizing the distribution of the numeric values.

2.3.4. **Determining the Best-Performing Team**

- **Data Preparation**: The data is read from "**results2.csv**", and the first row (representing overall statistics) is removed. The index is reset to ensure proper row alignment.
- **Criteria Selection and Scoring**: Only the columns containing **mean** values of each statistic are selected. For each of these, values are converted to numeric (non-numeric entries become **NaN**). The team with the highest average in each criterion is identified and awarded a point. These scores are tracked in a dictionary.
- **Final Evaluation**: After processing all criteria, the script prints which team topped each one. The team with the highest total score is declared the **best-performing team** of the 2024–2025 Premier League season.

2.4. Result Obtained

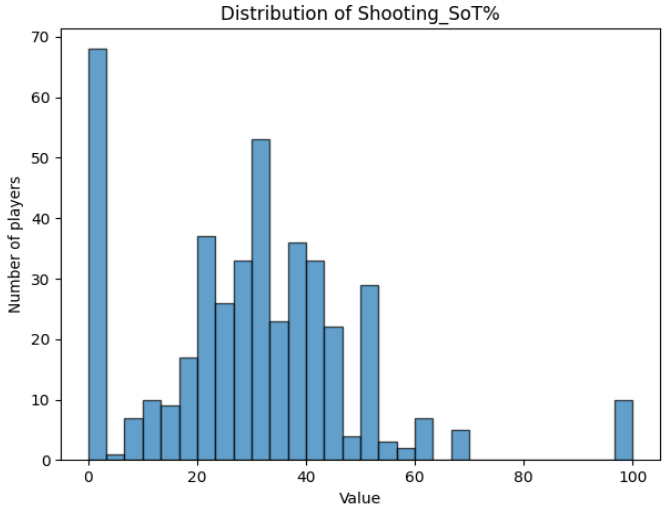
Overview of top_ 3.txt

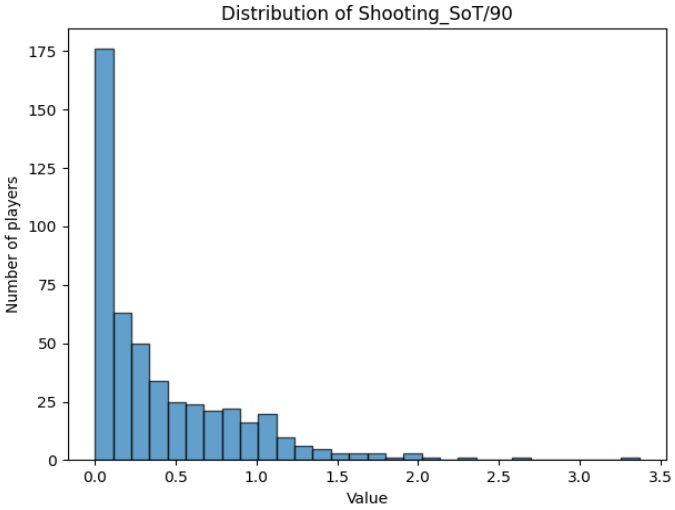
results2.csv

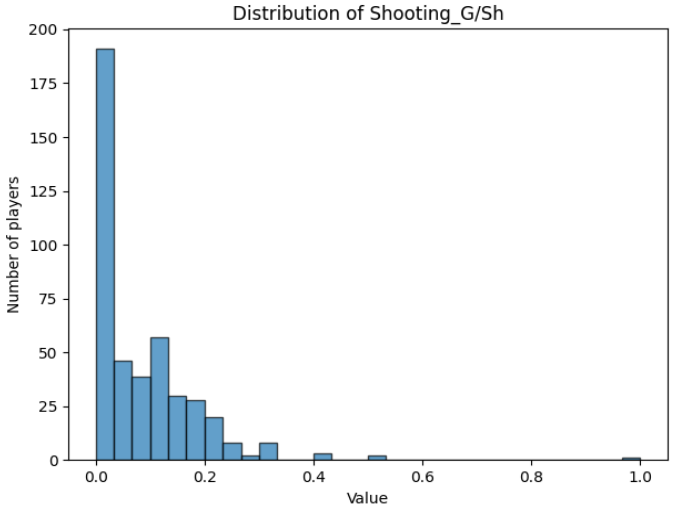
		Median of Age	Mean of Age	Std of Age	Median of Standard_MP	Mean of Standard_MP	Std of Stan
0	all	26.55	26.88	4.23	22.0	20.3	9.47
1	Arsenal	26.85	26.47	3.82	22.5	22.59	7.93
2	Aston Villa	27.6	27.11	4.04	20.0	18.86	9.97
3	Bournemouth	25.97	26.44	3.84	24.0	21.0	9.5
4	Brentford	24.81	26.17	3.91	26.0	22.24	10.84
5	Brighton	24.49	26.01	5.17	19.5	18.43	9.51
6	Chelsea	24.28	24.17	2.38	17.0	18.58	10.61
7	Crystal Palace	27.16	27.14	3.14	29.0	23.38	10.21
8	Everton	26.5	27.94	5.03	23.5	21.09	9.0
9	Fulham	28.8	28.79	3.35	25.0	23.09	9.07
10	Ipswich Town	26.68	26.87	3.15	18.0	17.17	8.64
11	Leicester City	26.72	27.19	4.46	21.0	19.15	9.35
12	Liverpool	26.42	27.18	3.69	27.0	23.71	8.56
13	Manchester City	26.66	26.97	4.91	22.0	19.24	8.76

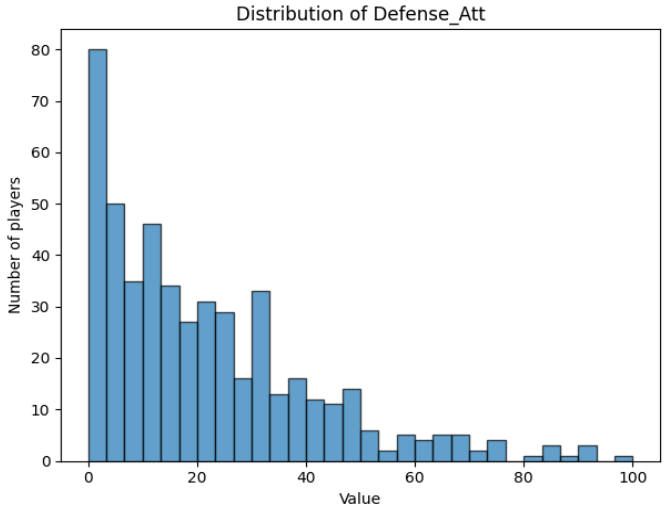
Overview of results2.csv

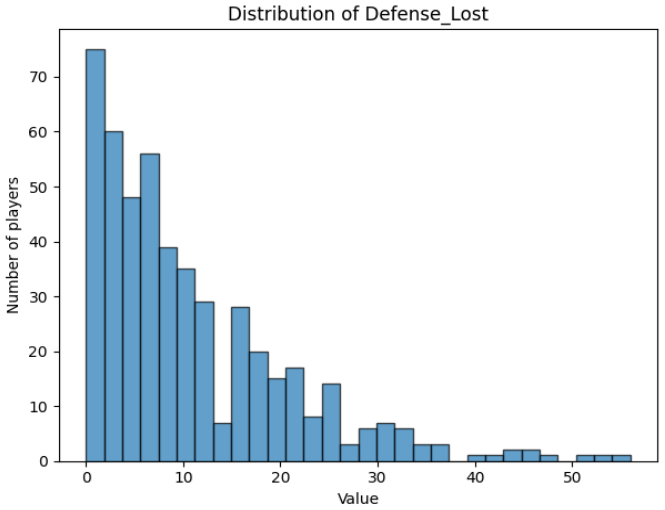
The result obtained includes two additional output files: **top_3.txt** and **result2.csv**. The **top_3.txt** file contains the top 3 and bottom 3 players for each statistic, providing a quick overview of the best and worst performers in each category. Meanwhile, the **result2.csv** file includes aggregated team-level data, with each team's median, mean, and standard deviation for every stat.

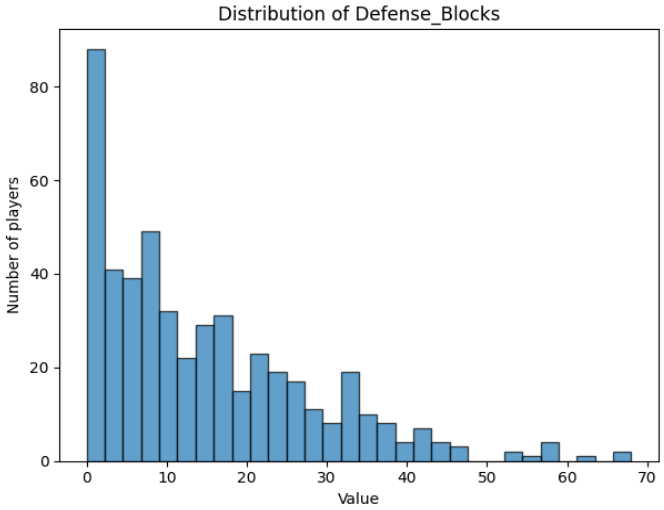




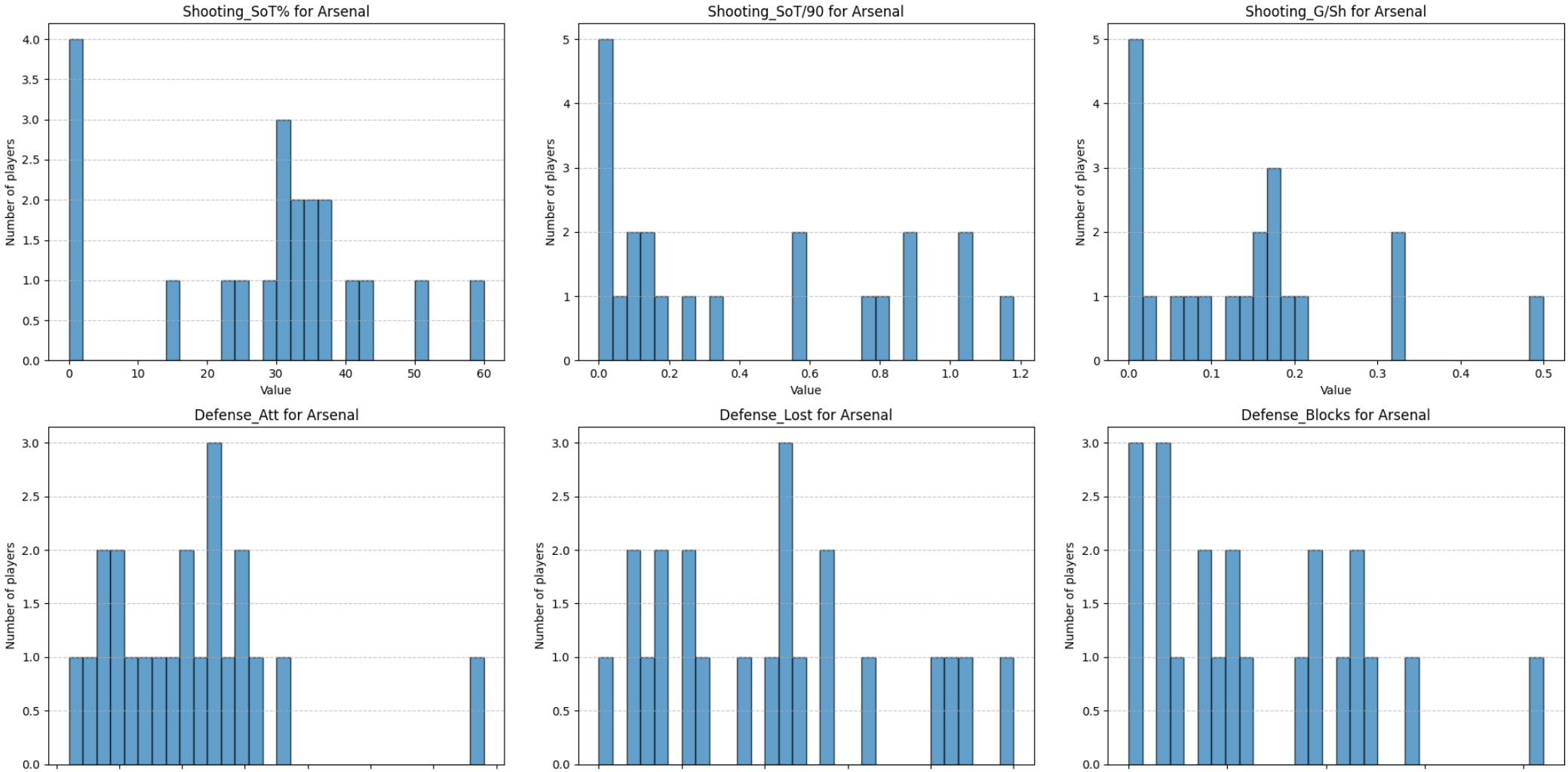








Histogram showing selected statistics of all players



Histogram showing selected statistics of Arsenal

The resulted histograms were created using the dataset and visualized with **Matplotlib** to illustrate the distribution of players across selected statistics: **Shooting_SoT%, Shooting_SoT/90, Shooting_G/Sh**, Defense_**Att, Defense_Lost**, and **Defense_Blocks**. These histograms offer a clear and intuitive depiction of how players are spread across these statistics, providing valuable insights into the overall performance distribution. By plotting the distribution for both all players and individual teams, the histograms enable a detailed comparison of player performance within each team as well as across the league. This visual approach makes it easier to spot trends, such as whether a specific stat is concentrated in a narrow range or spread more broadly, and enhances the understanding of the performance landscape for each of the selected metrics.

```
Team with the highest Misc_Lost is Crystal Palace
Team with the highest Misc_Won% is Southampton
Score check
Brentford: 5 scored
Nott'ham Forest: 4 scored
Wolves: 0 scored
Tottenham: 0 scored
Liverpool: 22 scored
Brighton: 0 scored
Manchester Utd: 0 scored
Arsenal: 3 scored
Newcastle Utd: 2 scored
Crystal Palace: 11 scored
Chelsea: 2 scored
Everton: 1 scored
Manchester City: 12 scored
Fulham: 3 scored
West Ham: 0 scored
Ipswich Town: 0 scored
Bournemouth: 4 scored
Leicester City: 2 scored
Aston Villa: 2 scored
Southampton: 1 scored
The best-performing team in the 2024-2025 Premier League season is Liverpool
```

Find the best-performing team in the Premier League

Based on the highest values for each statistic, Liverpool was identified as the best-performing team, which aligns with their current position as the top-ranked team on the scoreboard. This result is consistent with the team's strong performance

across various key metrics, reinforcing the idea that their high rankings are supported by superior statistics in areas such as shooting accuracy, defense, and overall gameplay. Given their current standings, this analysis further validates Liverpool's dominance in the league, confirming that their performance metrics are contributing factors to their success.

3. Player Clustering Using K-means and PCA

3.1. Objective

The objective of this problem is to apply the K-means clustering algorithm to classify players based on their statistics and determine the optimal number of clusters. After deciding on the number of clusters, Principal Component Analysis (PCA) is used to reduce the data dimensions to 2 for easier visualization. Finally, a 2D scatter plot is generated to display the clusters, allowing for an analysis of how players group together based on their performance data. The goal is to interpret the clustering results and gain insights into player similarities.

3.2. Tools **Used**

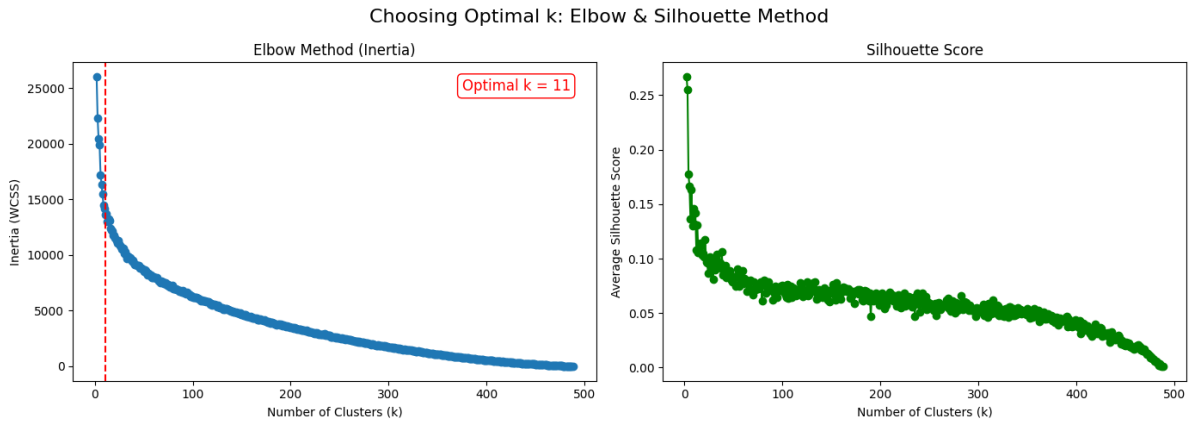
Library/Modules	**Type**	**Purpose**
Pandas	Third-party	Data manipulation and analysis, specifically loading, selecting, and handling data.
numpy	Third-party	Numerical operations and array manipulation.
matplotlib.pyplot	Third-party	Plotting and visualization of data, specifically creating scatter plots for clustering results.

sklearn.decomposition.PCA	Third-party	Principal Component Analysis (PCA) for dimensionality reduction.
sklearn.cluster.KMeans	Third-party	K-Means clustering algorithm for grouping the data into clusters.
sklearn.preprocessing.StandardScaler	Third-party	Standardizing features (scaling the data to have zero mean and unit variance).
sklearn.metrics.silhouette_score	Third-party	Calculating silhouette score to evaluate clustering performance.
kneed.KneeLocator	Third-party	Finding the optimal number of clusters using the elbow method (KneeLocator).
plotly.express	Third-party	Creating interactive plots and visualizations, used here for scatter plots.

3.3. Summary of Methodology

- **Data Preprocessing:** Relevant numeric columns were selected, non-numeric values were coerced, and missing data was filled with zeros. Features were standardized using `StandardScaler`.
- **Clustering:** KMeans clustering was applied on the scaled data. Optimal number of clusters (k) was determined using the **Elbow Method** (with `KneeLocator`) and validated using **Silhouette Score**.
- **Visualization:** PCA was used to reduce dimensions for exploratory visualization. Both **Matplotlib (Static)** and **Plotly (Interactive)** were used to plot PCA results.

3.4. Result Obtained



Elbow graph and silhouette graph to determine the optimal value of k for K-means

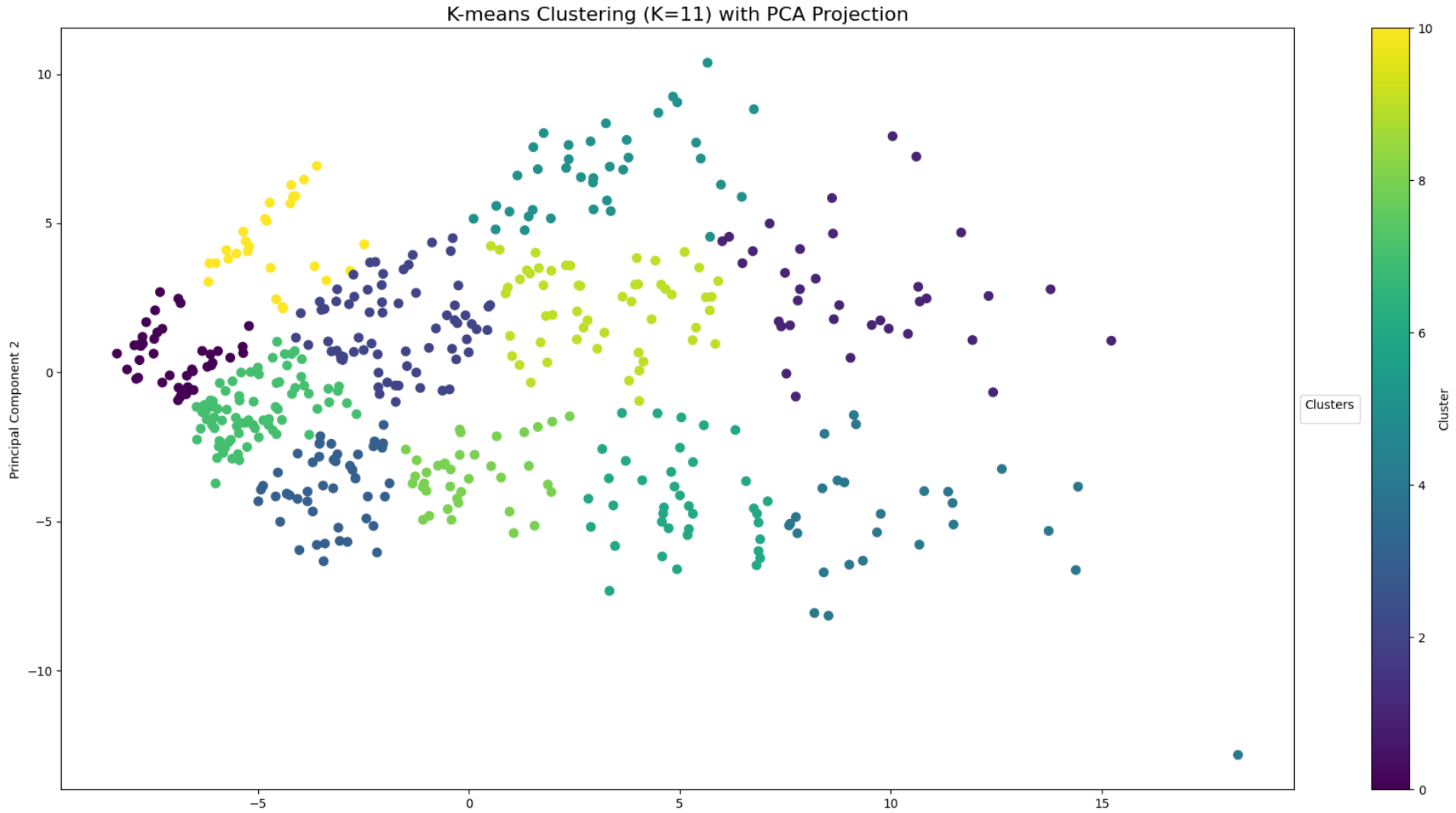
**How many groups should the players be classified into? Why? Provide your comments on the results*

In this analysis, the K-means clustering algorithm was applied to classify the data into distinct groups. The elbow method, combined with the kneeLocator tool, indicated that the optimal number of clusters (k) is 11. However, for practical football analysis, 11 clusters may still be excessive due to overlapping roles (e.g., Clusters 3 & 6, 4 & 8) and niche groups (e.g., Clusters 2 & 7), which could be merged into 6–8 clusters for better interpretability without losing key distinctions. The current 11 clusters effectively capture nuanced roles using comprehensive metrics (xG, progressive passes), but their complexity and potential omission of roles like goalkeepers or centre-backs limit applicability. To align with k-means while improving utility, consider merging similar clusters or validating with domain knowledge to ensure tactical relevance.

Cluster	Role	Key Traits
0	Young Attacking Midfielders/Wingers	High xG, progressive carries, chance creation

1	Defensive Midfield Anchors	Strong defending, reliable passing, low attack
2	Low-Minute Substitutes	Minimal playtime, low impact
3	Elite All-Around Midfielders	High attack, defense, and progressive play
4	Balanced Full-Backs	Moderate attack and defense, overlapping runs
5	Creative Playmakers	High assists, xAG, chance creation
6	High-Work-Rate Midfielders	Balanced attack/defense, high pressing
7	Veteran Bench Players	Defensive focus, minimal attack
8	Attacking Full-Backs/Wing-Backs	High crossing, assists, moderate defense
9	Goal-Scoring Forwards	High goals, xG, low defense
10	Utility Midfielders	Rotational, balanced but unspectacular

After applying Principal Component Analysis (PCA) for visualization, the results revealed 11 distinct clusters, each clearly separated from one another, with no significant overlap or clumping. This suggests that the dataset contains well-defined groups of players or teams based on their performance metrics. The clear separation between clusters indicates that different players or teams exhibit unique performance characteristics, and these differences are captured effectively by the PCA transformation. This outcome highlights the usefulness of PCA in reducing the dimensionality of complex data and revealing underlying patterns and structures within the dataset.



PCA graph of player data created using Matplotlib

*For more details, the interactive graph allows users to explore data points for additional information.

4. Player Transfer Value Prediction

4.1. Objective

The objective of this task is to scrape the transfer values of football players for the 2024-2025 season from FootballTransfers.com, focusing on players who have played more than 900 minutes. Using the collected data, a method will be developed to predict player transfer values. This method will involve selecting relevant features using machine learning techniques, such as feature importance or recursive feature elimination, and applying a suitable regression model, such as linear regression or decision tree, to estimate transfer values based on these selected features.

4.2. Tools Used

Libraries/Modules	Type	Purpose
requests	Third-party	Used to send HTTP POST requests to the API endpoint to fetch data.
pandas	Third-party	Used for data manipulation, merging, and saving data (like reading/writing CSV files).
unidecode	Third-party	Used to remove special characters from player names.
utils	Custom	Provides helper functions like external_lookup.
RandomForestRegressor	Third-party	Trains a machine learning model to predict continuous values (e.g., player transfer values) based on features from the input dataset.
train_test_split	Third-	Divides the dataset into training data (used to

	party	train the model) and test data (used to evaluate the model's performance)
mean_absoluste_ error, r2_score	Third-party	Calculates the mean absolute error (MAE),computes the R^2 score (coefficient of determination).
matplotlib.pyplot	Third-party	Used to create the bar plot of the top 20 features' importances, showing how important each feature is for predicting the player's transfer value.
LinearRegression	Third-party	Implements a linear regression model that is used for predicting continuous values based on linear relationships between features.
Ridge	Third-party	Implements Ridge Regression, which is a form of linear regression that includes L2 regularization to prevent overfitting.
Lasso	Third-party	Implements Lasso Regression, a linear model with L1 regularization, which can be used to automatically select important features by shrinking some coefficients to zero.
xgboost	Third-party	Implements XGBoost, a gradient boosting algorithm that excels in predictive accuracy and handling large datasets. It's used for building and training the regression model.
xgboost.plot_imp ortance	Third-party	Visualizes the importance of features based on the trained XGBoost model. It helps in understanding which features contribute most to the predictions.

xgboost.XGBRegressor	Third-party	Implements the XGBoost regression model, which uses gradient boosting for predictive tasks. It's tuned in this code to predict the target variable using important features.

4.3. Summary of Methodology

4.3.1. **Collect 2024–2025 player transfer values from [footballtransfers.com](), filtering only those with over 900 minutes of playtime.**

- **Scrape Transfer Data:** Fetch player transfer value data from **FootballTransfers** across multiple pages using **POST** requests and store selected columns.
- **Load Match Data:** Read local match stats (e.g., minutes played) from **results.csv.**
- **Merge Datasets:** Merge transfer values with match data on player names, keeping only those who played more than 900 minutes.
- **Fill Missing Values:** For unmatched players, perform **name normalization** and use `external_lookup()` to find transfer values manually.
- **Save Final Dataset:** Export the cleaned and enriched dataset `Transfer_values_unfiltered.csv`.

4.3.2. **Propose a method for estimating player values**

Collecting data: Data collection involves reading two CSV files into DataFrames and merging them with a left join on "Player" and "Team" to preserve all transfer records. Missing values marked as "N/a" are replaced with pd.NA for consistency.

Data Preprocessing: The preprocessing begins by identifying and listing object and numeric columns. Selected performance-related columns are then converted to numeric, handling any errors by coercing invalid values to NaN. A custom function is applied to convert currency-formatted strings in the "Estimated Value" column into numeric values. The dataset is further cleaned by removing outliers and dropping irrelevant or redundant columns. Finally, all missing values are filled with 0 to ensure consistency in subsequent analysis.

Data Visualization: The data visualization focuses on identifying the top 20 features most strongly correlated with the target variable, "Estimated Value." After computing the absolute correlations, these features are extracted and used to generate a correlation matrix. A heatmap is then plotted using Seaborn to visually display the strength and direction of relationships between the selected features and the target, helping highlight key predictors for further analysis.

Choosing Model: The model selection process involves training five regression models—Linear, Ridge, Lasso, Random Forest, and XGBoost—on a dataset split into training, validation, and test sets. Each model is evaluated using Mean Squared Error (MSE), Mean Absolute Error (MAE), and R^2 score on the test set, while validation R^2 is used for additional insight. The performance metrics are visualized using bar plots, enabling a clear comparison to identify the most accurate and reliable model for predicting estimated player values. After evaluating the models, Linear Regression is chosen since it has the highest R^2 score and a decent MAE and MSE.

Building Model: The model was built by filtering relevant features, handling missing values, and applying a log transformation to the target variable. After splitting the data into training and test sets, features were standardized and a linear regression model was trained. Performance was evaluated using R^2 scores and cross-validation MSE. Predictions were inverse-transformed to the original scale, with

metrics (MSE, MAE, R^2) calculated and a scatter plot generated to visualize actual versus predicted values on the test set.

4.4. Result Obtained

4.4.1. **Collecting player transfer value data**

The final transfer value file contains all players who played more than 900 minutes, as listed in the **results.csv** file. During processing, differences in player name formats between the two source websites were handled to ensure accurate matching. The dataset also includes players who left the league but still appear in results.csv, allowing for a complete historical record.

	Player	Team	Played Time	Estimated Value
2	Aaron Ramsdale	Southampton	2340	€18.7M
3	Aaron Wan-Bissaka	West Ham	2794	€29.7M
4	Abdoulaye Doucouré	Everton	2425	€5.8M
5	Adam Armstrong	Southampton	1248	€15.1M
6	Adam Smith	Bournemouth	1409	€1.5M
7	Adam Wharton	Crystal Palace	1258	€49.3M
8	Adama Traoré	Fulham	1568	€8.2M
9	Alejandro Garnacho	Manchester Utd	2056	€59.2M
10	Alex Iwobi	Fulham	2721	€31.2M
11	Alexander Isak	Newcastle Utd	2487	€119.4M
12	Alexis Mac Allister	Liverpool	2553	€117M
13	Alisson	Liverpool	2148	€24.5M
14	Alphonse Areola	West Ham	1990	€11.6M
15	Amad Diallo	Manchester Utd	1594	€48.6M
16	Amadou Onana	Aston Villa	1348	€64.4M
17	Andreas Pereira	Fulham	1879	€18.9M
18	Andrew Robertson	Liverpool	2308	€20.6M
19	André	Wolves	2170	€29.4M
20	André Onana	Manchester Utd	2970	€34.2M
21	Anthony Elanga	Nott'ham Forest	2052	€45.6M
22	Anthony Gordon	Newcastle Utd	2200	€52.4M

4.4.2. **Developing a machine learning model to predict player transfer value**

```python
import pandas as pd
player_df = pd.read_csv("D:\\Python-Assigment\\source code\\results.csv")
transfer_df = pd.read_csv("D:\\Python-Assigment\\source code\\Transfer_values.csv")


#merge 2 df on
result_df = pd.merge(transfer_df,player_df,how="left", on=["Player","Team"])
result_df.replace("N/a",pd.NA,inplace=True)
#Show all rows
print(result_df.info())
```

✓ 0.0s

```
53  Defense_Int                299 non-null    int64
54  Possession_Touches         299 non-null    int64
55  Possession_Def Pen         299 non-null    int64
56  Possession_Def 3rd         299 non-null    int64
57  Possession_Mid 3rd         299 non-null    int64
58  Possession_Att 3rd         299 non-null    int64
59  Possession_Att Pen         299 non-null    int64
60  Possession_Att             299 non-null    int64
61  Possession_Succ%           284 non-null    object
```

Information about the dataset after loading and merging into a single DataFrame

```python
numeric_columns = result_df.select_dtypes(include='number').columns.tolist()
print("Numeric columns:", numeric_columns)
result_df
```

✓ 0.0s

```
Number of object columns: 0
Object columns: []
Number of numeric columns: 70
Numeric columns: ['Played Time', 'Estimated Value', 'Age', 'Standard_MP', 'Standard_Starts', 'Standard_Min', 'Standard_Gls', 'Standard_Ast', 'Standard_
```

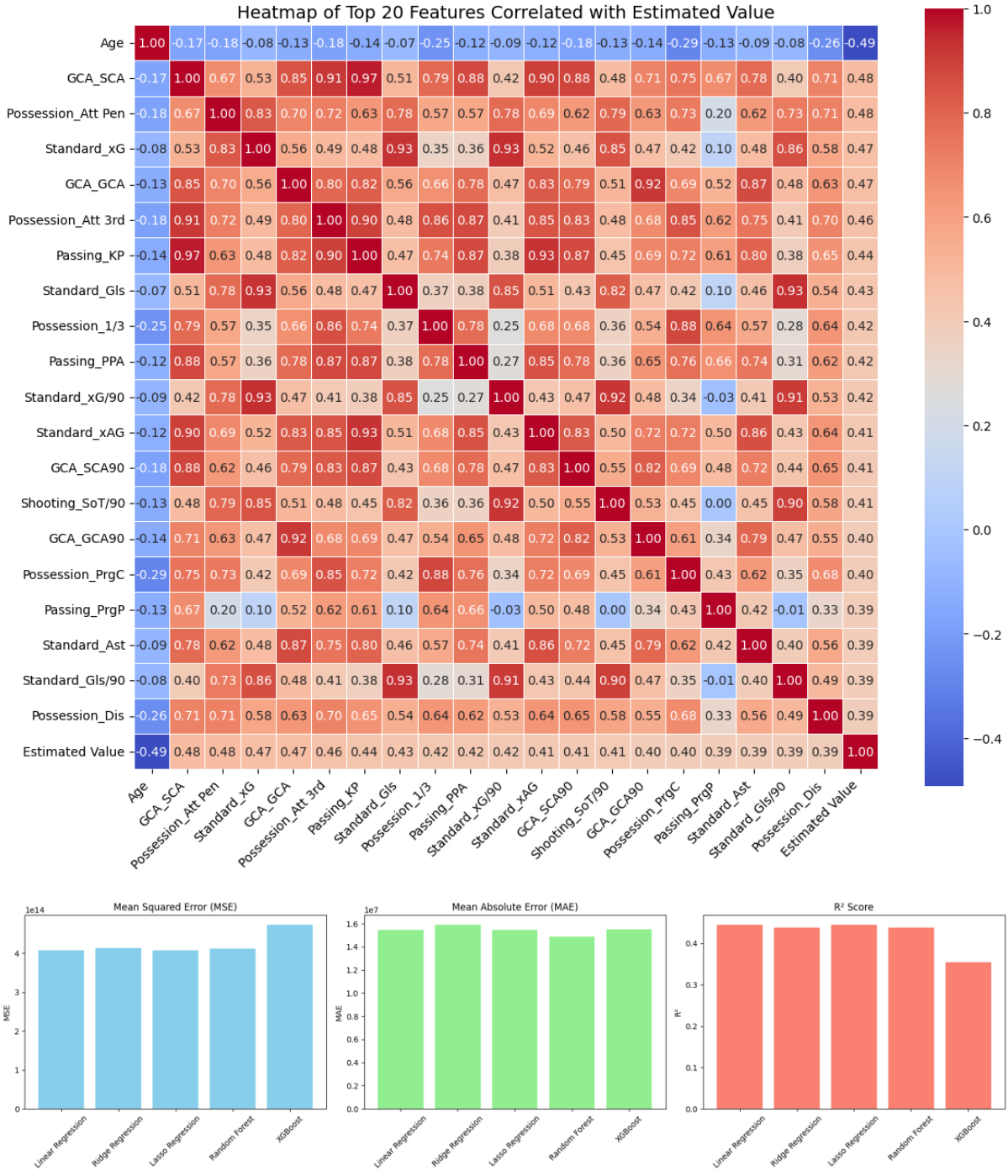
	Played Time	Estimated Value	Age	Standard_MP	Standard_Starts	Standard_Min	Standard_Gls	Standard_Ast	Standard_CrdY	Standard_CrdR	...	Possession_Rec	Possession_PrgR	Misc_Fls	M
0	2340.0	18700000.0	26.96	26	26	2340.0	0	0	2	0	...	628	0	1	
1	2794.0	29700000.0	27.42	32	31	2794.0	2	2	1	0	...	1135	139	22	
2	2425.0	5800000.0	32.33	30	29	2425.0	3	1	5	1	...	705	91	46	
3	1248.0	15100000.0	28.22	20	15	1248.0	2	2	4	0	...	286	79	12	
4	1409.0	1500000.0	34.00	22	17	1409.0	0	0	6	0	...	333	31	15	
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	
294	1467.0	15800000.0	24.84	29	15	1467.0	1	1	4	0	...	692	76	15	
295	1210.0	30000000.0	28.67	24	15	1210.0	2	0	7	0	...	657	3	22	
296	955.0	11200000.0	31.89	15	11	955.0	0	1	2	0	...	258	34	12	
297	1967.0	11000000.0	34.52	30	22	1967.0	0	5	1	0	...	1330	81	14	
298	1070.0	900000.0	40.03	13	12	1070.0	0	0	2	0	...	227	0	2	

298 rows × 70 columns

After data processing

```
 Top 20 features with highest absolute correlation to the target variable:
Age ⟶ Correlation: 0.4926
GCA_SCA ⟶ Correlation: 0.4849
Possession_Att Pen ⟶ Correlation: 0.4830
Standard_xG ⟶ Correlation: 0.4685
GCA_GCA ⟶ Correlation: 0.4684
Possession_Att 3rd ⟶ Correlation: 0.4558
Passing_KP ⟶ Correlation: 0.4363
Standard_Gls ⟶ Correlation: 0.4338
Possession_1/3 ⟶ Correlation: 0.4215
Passing_PPA ⟶ Correlation: 0.4181
Standard_xG/90 ⟶ Correlation: 0.4158
Standard_xAG ⟶ Correlation: 0.4143
GCA_SCA90 ⟶ Correlation: 0.4137
Shooting_SoT/90 ⟶ Correlation: 0.4064
GCA_GCA90 ⟶ Correlation: 0.3987
Possession_PrgC ⟶ Correlation: 0.3974
Passing_PrgP ⟶ Correlation: 0.3928
Standard_Ast ⟶ Correlation: 0.3914
Standard_Gls/90 ⟶ Correlation: 0.3901
Possession_Dis ⟶ Correlation: 0.3860
```

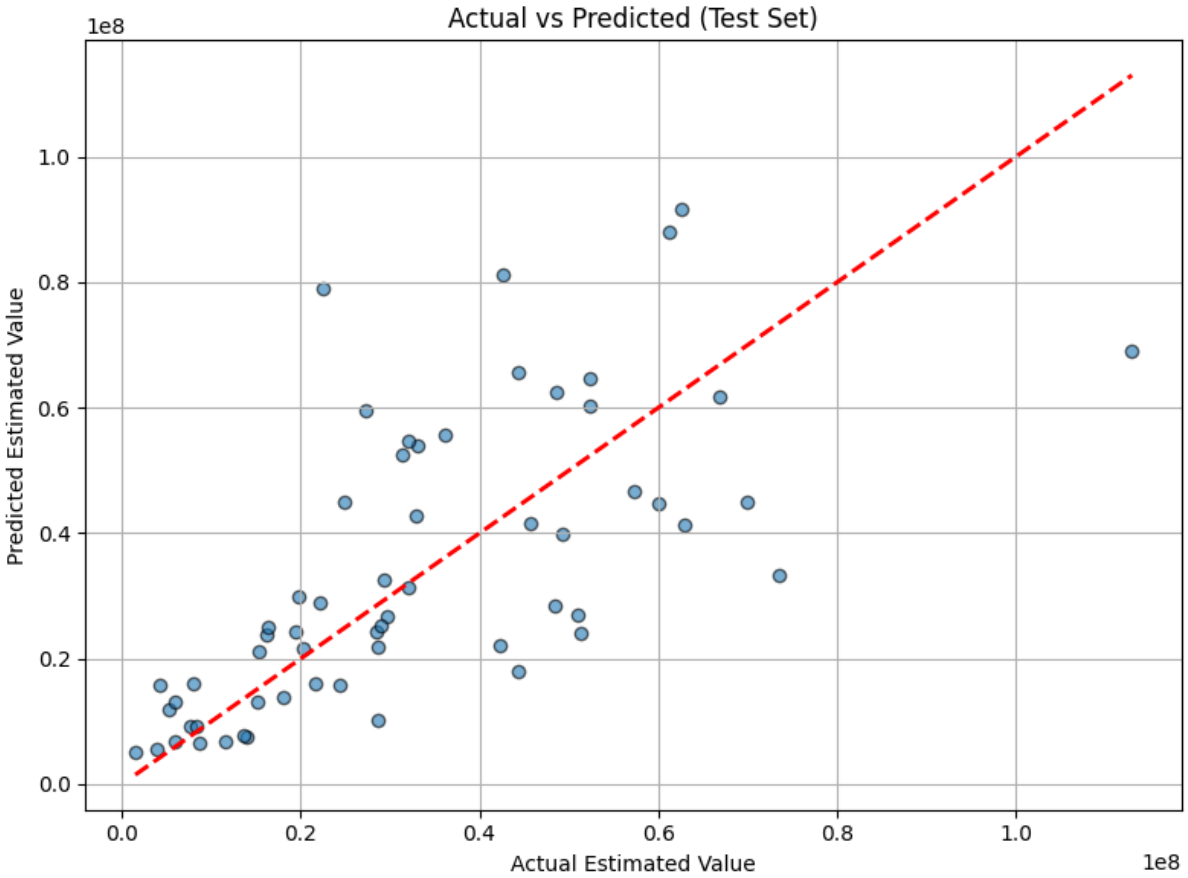
20 features with highest absolute correlation to the target variable



Choosing Model

```
    plt.grid(True)
    plt.tight_layout()
    plt.show()

✓ 0.1s

Data split into training and testing sets.
Linear Regression training completed.

Model Performance:
Training R²: 0.6034
Test R²: 0.6093

Cross-validation MSE on training set: -0.4637643522315796

Training Metrics:
  MSE : 396539195983729.6875
  MAE : 13125385.8330
  R²  : 0.6034

Test Metrics:
  MSE : 328716983341225.0000
  MAE : 13554540.1071
  R²  : 0.6093
```

Performance of the Linear Regression model



Actual vs. predicted values for the Linear Regression model

Linear Regression model shows moderate performance, with a training R^2 of 0.6034, meaning it explains approximately 60.34% of the variance in the training data. While this indicates some level of fit, the model still leaves room for improvement. The Mean Squared Error (MSE) on the training set is quite high at 396,539,195,983,729.69, suggesting significant prediction errors, while the Mean Absolute Error (MAE) of 13,125,385.83 shows that, on average, the predictions are off by over 13 million units. On the test set, the model performs similarly, with a test R^2 of 0.6093, explaining about 60.93% of the variance. The MSE on the test set is slightly smaller (328,716,983,341,225.00), indicating better generalization to unseen data, though the error is still substantial. The MAE on the test set is also similar to the training set, indicating consistent performance but still showing significant prediction errors. Additionally, the cross-validation MSE on the training set is -0.4638, which suggests the model might underperform compared to a baseline model in certain cross-validation folds. In conclusion, while the model demonstrates some predictive power, its high MSE and MAE values indicate that further improvements through feature engineering or model tuning are needed for better accuracy.

5. Conclusion

In this assignment, we developed a comprehensive data analysis pipeline for evaluating football player performance in the 2024–2025 English Premier League season. Our Python program successfully collected and processed detailed player statistics from FBref, focusing on those with more than 90 minutes of playtime. The data was cleaned, standardized, and saved in CSV format for further analysis.

We identified the top and bottom performers across a wide range of metrics and performed statistical analysis to compute the median, mean, and standard deviation of each attribute—both league-wide and by individual teams. Visualization through histograms provided deeper insight into the distribution of these statistics, highlighting performance trends across players and clubs.

Using unsupervised machine learning techniques, we applied K-means clustering to group players by similarity in performance. Dimensionality reduction using PCA enabled us to visualize the clustering in two dimensions, which revealed distinguishable patterns in player roles and styles.

Finally, we collected transfer values from a secondary source and proposed a predictive approach for estimating player market value based on statistical features and machine learning regression models.

This project demonstrates the power of data science in sports analytics and provides a foundation for more advanced performance modeling and player valuation systems in the future.

6.Acknowledgement

I would like to sincerely thank my instructor for providing guidance and clear instructions throughout this assignment.

I also appreciate the resources from FBref.com and FootballTransfers.com, which made the data collection process possible.

Additionally, I would like to thank the Python open-source community for providing libraries such as pandas, matplotlib, and scikit-learn, which were essential for data analysis and visualization.

Finally, I am grateful for the support from my classmates and online communities during the completion of this project.